**به نام خدا**

**دانشگاه تهران**
**دانشکدهٔ مهندسی برق و کامپیوتر**

# ساختار داده و الگوریتم

پاییز ۱۴۰۶–۱۴۰۵
مدرس: دکتر نگار آقاپور صباغ
نام دانشجو: امیرعلی حاجی زاده مقدم
شماره دانشجویی: ۸۱۰۱۰۲۴۳۰

تمرین دوم

---

## پاسخ سؤالات نظری

---

### سؤال ۱

**پاسخ:**

**ایدهٔ حل:**

چون کلمات هر فایل از قبل به ترتیب الفبایی مرتب شده‌اند، برای ادغام آن‌ها لازم نیست همهٔ کلمات را دوباره مرتب کنیم. کافی است در هر مرحله کوچک‌ترین کلمهٔ باقی‌مانده را انتخاب کنیم. برای اینکه مجبور نباشیم هر بار اولین کلمهٔ همهٔ $k$ فایل را بررسی کنیم، از Min-Heap استفاده می‌کنیم.

ابتدا اولین کلمهٔ هر فایل غیرخالی را همراه با شمارهٔ فایل در هیپ قرار می‌دهیم. در نتیجه در هر لحظه حداکثر یک کلمه از هر فایل داخل هیپ داریم.

**مراحل الگوریتم:**

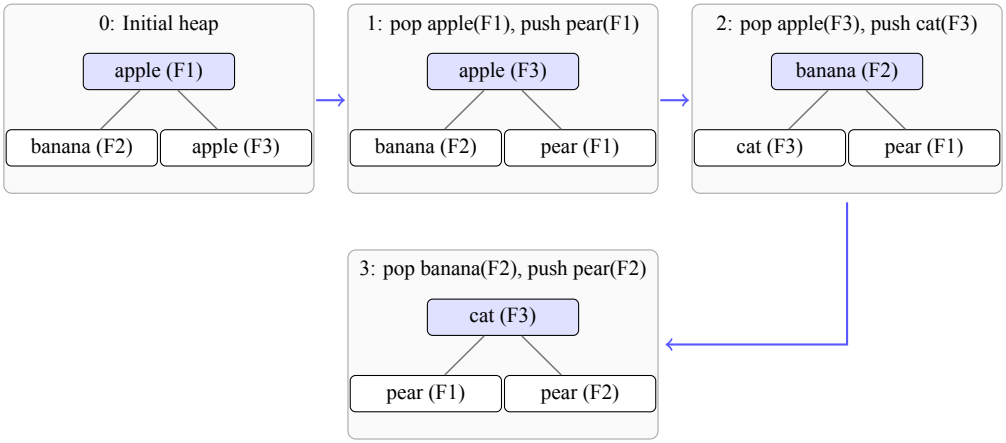
کوچک‌ترین کلمه را از ریشهٔ هیپ خارج می‌کنیم. اگر با آخرین کلمهٔ ذخیره‌شده در خروجی متفاوت باشد، آن را به خروجی اضافه می‌کنیم. سپس، مستقل از اینکه کلمه تکراری بوده یا نه، کلمهٔ بعدی **همان فایل** را وارد هیپ می‌کنیم. اگر آن فایل تمام شده باشد، چیزی اضافه نمی‌شود. بعد از درج، هیپ دوباره ویژگی Min-Heap را برقرار می‌کند. این کار را تا خالی شدن هیپ ادامه می‌دهیم.

**بررسی مثال:**

برای مشخص شدن نحوهٔ به‌روزرسانی هیپ، سه فایل زیر را در نظر می‌گیریم:

| | |
|---|---|
| File 1 | apple, pear |
| File 2 | banana, pear |
| File 3 | apple, cat |

در شکل زیر، چهار وضعیت اول هیپ نشان داده شده است. هر گره شامل کلمه و شمارهٔ فایل آن است. پس از خارج کردن ریشه، کلمهٔ بعدی همان فایل وارد می‌شود و هیپ دوباره تنظیم می‌شود. برای دو کلمهٔ برابر، در این مثال شمارهٔ فایل کوچک‌تر را زودتر انتخاب کرده‌ایم.




همهٔ مراحل استخراج و درج را می‌توان به صورت زیر دنبال کرد. در ستون «هیپ پس از به‌روزرسانی»، کلمات فقط برای خوانایی به ترتیب الفبایی نوشته شده‌اند؛ شکل بالا ساختار درختی هیپ را نشان می‌دهد.

| Step | Extract | Insert from same file | Heap after update |
|---|---|---|---|
| 1 | apple (F1) | pear (F1) | apple(F3), banana(F2), pear(F1) |
| 2 | apple (F3) | cat (F3) | banana(F2), cat(F3), pear(F1) |
| 3 | banana (F2) | pear (F2) | cat(F3), pear(F1), pear(F2) |
| 4 | cat (F3) | – | pear(F1), pear(F2) |
| 5 | pear (F1) | – | pear(F2) |
| 6 | pear (F2) | – | empty |

کلمهٔ apple در مرحلهٔ دوم و pear در مرحلهٔ ششم تکراری هستند؛ پس فقط یک بار در خروجی ذخیره می‌شوند. خروجی نهایی برابر است با:

apple, banana, cat, pear

**دلیل درستی الگوریتم:**

چون هر فایل مرتب است، اولین کلمهٔ باقی‌ماندهٔ آن از بقیهٔ کلمات همان فایل کوچک‌تر یا مساوی است. در هیپ نیز اولین کلمهٔ همهٔ فایل‌های تمام‌نشده را داریم. پس ریشهٔ هیپ همیشه کوچک‌ترین کلمهٔ باقی‌مانده بین تمام فایل‌هاست و ترتیب الفبایی خروجی حفظ می‌شود. در نتیجه کلمات یکسان پشت سر هم استخراج می‌شوند و مقایسه با آخرین کلمهٔ خروجی برای حذف تکراری‌ها کافی است.

**پیچیدگی زمانی و حافظه:**

هر کلمه یک بار وارد هیپ و یک بار از آن خارج می‌شود. اندازهٔ هیپ در هیچ مرحله‌ای بیشتر از $k$ نیست، پس هر درج و استخراج $O(\log k)$ زمان می‌برد. با توجه به مجموع $n$ کلمه، زمان اجرای الگوریتم برابر است با:

$$T(n, k) = O(n \log k)$$

برای نگهداری هیپ به $O(k)$ حافظهٔ اضافی نیاز داریم؛ فضای لیست خروجی جدا از این مقدار است.

**سؤال ۲**

**پاسخ:**

**ایدهٔ حل:**

در این سؤال آرایه‌ای شامل $n$ عدد صحیح داریم که مقادیر آن‌ها در بازهٔ $[0, n^2 - 1]$ قرار دارند. هدف این است که آرایه را در زمان $O(n)$ مرتب کنیم.

در روش Counting Sort، تعداد تکرار هر مقدار را می‌شماریم و بر اساس آن آرایه را مرتب می‌کنیم. اما Radix Sort به جای مرتب کردن مستقیم مقادیر، اعداد را رقم‌به‌رقم مرتب می‌کند. در هر مرحله نیز می‌توان از Counting Sort پایدار برای مرتب کردن یک رقم استفاده کرد.

**الف)**

**مراحل الگوریتم:**

برای استفاده از Radix Sort، ابتدا اعداد را در مبنای $n$ در نظر می‌گیریم. دلیل انتخاب این مبنا آن است که بزرگ‌ترین عدد ممکن برابر $n^2 - 1$ است و تمام اعداد در این مبنا حداکثر دو رقم خواهند داشت.

هر عدد $x$ را می‌توان به صورت زیر نوشت:

$$x = d_1 n + d_0$$

که در آن رقم کم‌ارزش و رقم پرارزش به ترتیب از روابط زیر محاسبه می‌شوند:

$$d_0 = x \bmod n \qquad d_1 = \left\lfloor \frac{x}{n} \right\rfloor$$

چون $0 \le x < n^2$ است، هر دو رقم در بازهٔ $[0, n-1]$ قرار می‌گیرند.

بنابراین برای مرتب کردن آرایه فقط به دو مرحله مرتب‌سازی نیاز داریم:

۱. در مرحلهٔ اول، تمام اعداد را بر اساس رقم کم‌ارزش $d_0$ به صورت صعودی مرتب می‌کنیم.

۲. در مرحلهٔ دوم، آرایهٔ حاصل را بر اساس رقم پرارزش $d_1$ مرتب می‌کنیم.

برای هر دو مرحله از Counting Sort پایدار استفاده می‌کنیم.

در Counting Sort ابتدا یک آرایهٔ شمارنده با $n$ خانه می‌سازیم؛ زیرا هر رقم فقط می‌تواند مقداری بین $0$ تا $n-1$ داشته باشد.

سپس تعداد تکرار هر رقم را می‌شماریم. با محاسبهٔ مجموع‌های تجمعی آرایهٔ شمارنده، جایگاه نهایی عناصر در خروجی مشخص می‌شود. در نهایت با پیمایش آرایهٔ ورودی از انتها به ابتدا و قرار دادن عناصر در جایگاه مناسب، مرتب‌سازی پایدار انجام می‌شود.

پایدار بودن مرتب‌سازی ضروری است؛ زیرا اگر دو عدد رقم پرارزش یکسانی داشته باشند، ترتیبی که در مرحلهٔ اول بر اساس رقم کم‌ارزش مشخص شده است، نباید در مرحلهٔ دوم تغییر کند.

**بررسی مثال:**

فرض می‌کنیم $n = 4$ و آرایهٔ ورودی به صورت زیر باشد:

$$A = [13, 2, 9, 5]$$

چون مبنا برابر $4$ است، هر عدد را به صورت $x = 4d_1 + d_0$ نمایش می‌دهیم:

$$13 = 3(4) + 1 = (31)_4$$
$$2 = 0(4) + 2 = (02)_4$$
$$9 = 2(4) + 1 = (21)_4$$
$$5 = 1(4) + 1 = (11)_4$$

بنابراین رقم‌های هر عدد به صورت زیر هستند:

| Number | Base 4 | $d_1$ | $d_0$ |
|---|---|---|---|
| 13 | 31 | 3 | 1 |
| 2 | 02 | 0 | 2 |
| 9 | 21 | 2 | 1 |
| 5 | 11 | 1 | 1 |

**مرحلهٔ اول: مرتب‌سازی بر اساس رقم کم‌ارزش**

رقم‌های کم‌ارزش اعداد در آرایهٔ اولیه به ترتیب برابر هستند با:

$$d_0 = [1, 2, 1, 1]$$

با اجرای Counting Sort پایدار بر اساس این رقم، ابتدا اعدادی که رقم کم‌ارزش آن‌ها 1 است قرار می‌گیرند و سپس عددی که رقم کم‌ارزش آن 2 است.

در نتیجه:

$$[13, 2, 9, 5] \longrightarrow [13, 9, 5, 2]$$

دقت می‌کنیم که چون رقم کم‌ارزش اعداد 13، 9 و 5 یکسان است، ترتیب اولیهٔ آن‌ها حفظ می‌شود.

**مرحلهٔ دوم: مرتب‌سازی بر اساس رقم پرارزش**

حالا آرایهٔ حاصل از مرحلهٔ اول را بر اساس رقم پرارزش مرتب می‌کنیم:

$$[13, 9, 5, 2]$$

رقم‌های پرارزش به ترتیب برابرند با:

$$d_1 = [3, 2, 1, 0]$$

پس با اجرای دومین Counting Sort پایدار، اعداد بر اساس رقم پرارزش به ترتیب 0، 1، 2 و 3 قرار می‌گیرند:

$$[13, 9, 5, 2] \longrightarrow [2, 5, 9, 13]$$

بنابراین آرایه پس از دو مرحله مرتب شده است.

برای جمع‌بندی، تغییر آرایه در هر مرحله به صورت زیر است:

| Step | Array |
|---|---|
| Initial | 13, 2, 9, 5 |
| Sort by $d_0$ | 13, 9, 5, 2 |
| Sort by $d_1$ | 2, 5, 9, 13 |

**دلیل درستی الگوریتم:**

بعد از مرحلهٔ اول، اعداد بر اساس رقم کم‌ارزش مرتب هستند.

در مرحلهٔ دوم، اعدادی که رقم پرارزش کوچک‌تری دارند، زودتر قرار می‌گیرند. اگر رقم پرارزش دو عدد برابر باشد، به دلیل پایدار بودن Counting Sort، ترتیب آن‌ها بر اساس رقم کم‌ارزش که در مرحلهٔ قبل به دست آمده است، حفظ می‌شود.

بنابراین ابتدا رقم پرارزش و در صورت برابر بودن آن، رقم کم‌ارزش ترتیب اعداد را تعیین می‌کند. این دقیقاً همان ترتیب صعودی مقادیر اصلی است.

از آنجا که تمام اعداد در مبنای $n$ حداکثر دو رقم دارند، بعد از همین دو مرحله آرایه به طور کامل مرتب می‌شود.

**پیچیدگی زمانی:**

در هر مرحله، تعداد عناصر ورودی $n$ است و رقم مورد بررسی نیز فقط $n$ مقدار ممکن دارد.

بنابراین ساخت آرایهٔ شمارنده، شمارش ارقام، محاسبهٔ مجموع تجمعی و تشکیل خروجی همگی در مجموع $O(n)$ زمان نیاز دارند.

پس پیچیدگی هر مرحله از Counting Sort برابر است با:

$$O(n+n) = O(n)$$

چون تعداد مراحل Radix Sort برابر دو و مستقل از $n$ است، پیچیدگی زمانی کل برابر می‌شود با:

$$\boxed{T(n) = 2O(n) = O(n)}$$

**ب)**

**بررسی Sort Counting مستقیم:**

در Counting Sort مستقیم، برای هر مقدار ممکن یک خانه در آرایهٔ شمارنده در نظر می‌گیریم.

سپس یک بار آرایهٔ ورودی را پیمایش می‌کنیم و تعداد تکرار هر مقدار را در خانهٔ مربوط به آن افزایش می‌دهیم. در ادامه با پیمایش آرایهٔ شمارنده، اعداد را به ترتیب صعودی در خروجی قرار می‌دهیم.

بنابراین پیچیدگی زمانی این روش به تعداد عناصر و اندازهٔ بازهٔ مقادیر ممکن وابسته است:

$$T(n) = O(n+K)$$

که $K$ اندازهٔ دامنهٔ اعداد است.

در این سؤال مقدار هر عنصر می‌تواند بین 0 و $n^2-1$ باشد. پس برای اجرای مستقیم Counting Sort به یک آرایهٔ شمارنده با $n^2$ خانه نیاز داریم.

حتی اگر فقط $n$ عدد در آرایهٔ ورودی وجود داشته باشد، باید این $n^2$ خانه را مقداردهی اولیه کرده و برای تولید خروجی بررسی کنیم.

بنابراین:

$$T(n) = O(n+n^2)$$

و در نتیجه:

$$\boxed{T(n) = O(n^2)}$$

پس با اجرای مستقیم Counting Sort استاندارد نمی‌توان این آرایه را در زمان $O(n)$ مرتب کرد.

تفاوت روش قسمت الف در این است که Radix Sort به جای در نظر گرفتن تمام $n^2$ مقدار ممکن، هر بار فقط یکی از دو رقم عدد را مرتب می‌کند.

چون هر رقم تنها $n$ مقدار ممکن دارد و تعداد مراحل نیز ثابت و برابر دو است، پیچیدگی کل در روش Radix Sort به $O(n)$ کاهش پیدا می‌کند.

## سؤال ۳

**پاسخ:**

**ایدهٔ حل:**

در این سؤال آرایه‌ای شامل $n$ عدد حقیقی در بازهٔ $[0, k)$ داریم و می‌خواهیم آن را با الگوریتم Bucket Sort مرتب کنیم.

ایدهٔ این الگوریتم این است که به جای مرتب کردن تمام اعداد به صورت یکجا، بازهٔ مقادیر را به چند قسمت تقسیم کنیم و هر عدد را در سطل مربوط به خودش قرار دهیم. سپس عناصر هر سطل را جداگانه مرتب کرده و در نهایت سطل‌ها را به ترتیب به یکدیگر متصل می‌کنیم.

**الف)**

**مراحل الگوریتم:**

ابتدا $n$ سطل خالی در نظر می‌گیریم و بازهٔ $[0, k)$ را به $n$ قسمت مساوی تقسیم می‌کنیم. بنابراین عرض هر سطل برابر است با:

$$w = \frac{k}{n}$$

سطل شمارهٔ $i$ شامل اعدادی در بازهٔ زیر خواهد بود:

$$\left[\frac{ik}{n}, \frac{(i+1)k}{n}\right)$$

برای تعیین شمارهٔ سطل هر عدد $x$ از رابطهٔ زیر استفاده می‌کنیم:

$$\boxed{index(x) = \left\lfloor \frac{nx}{k} \right\rfloor}$$

از آنجا که $0 \le x < k$ است، شمارهٔ سطل همواره بین 0 تا $n-1$ خواهد بود.

پس از تعیین سطل هر عدد، آن را در سطل مربوط قرار می‌دهیم. در این مرحله عناصر داخل هر سطل لزوماً مرتب نیستند.

سپس هر سطل را به صورت مستقل با الگوریتم Insertion Sort مرتب می‌کنیم. در نهایت، سطل‌ها را از شمارهٔ صفر تا $n-1$ پیمایش کرده و عناصر مرتب‌شدهٔ آن‌ها را به ترتیب در آرایهٔ خروجی قرار می‌دهیم.

**بررسی مثال:**

فرض می‌کنیم آرایهٔ ورودی به صورت زیر باشد:

$$A = [7.2, 1.5, 8.1, 3.4, 1.2, 6.7]$$

در این مثال $n = 6$ و $k = 10$ است. بنابراین شش سطل با عرض زیر داریم:

$$w = \frac{10}{6} = \frac{5}{3}$$

شمارهٔ سطل هر عدد با استفاده از رابطهٔ $\lfloor 6x/10 \rfloor$ محاسبه می‌شود:

$$7.2 \longrightarrow 4$$
$$1.5 \longrightarrow 0$$
$$8.1 \longrightarrow 4$$
$$3.4 \longrightarrow 2$$
$$1.2 \longrightarrow 0$$
$$6.7 \longrightarrow 4$$

در نتیجه، وضعیت سطل‌ها قبل و بعد از مرتب‌سازی به صورت زیر خواهد بود:

| Bucket | Before sorting | After sorting |
|---|---|---|
| 0 | 1.5, 1.2 | 1.2, 1.5 |
| 1 | – | – |
| 2 | 3.4 | 3.4 |
| 3 | – | – |
| 4 | 7.2, 8.1, 6.7 | 6.7, 7.2, 8.1 |
| 5 | – | – |

پس از مرتب کردن عناصر هر سطل، سطل‌ها را از صفر تا پنج به ترتیب به یکدیگر متصل می‌کنیم و خروجی زیر به دست می‌آید:

$$\boxed{A_{\text{sorted}} = [1.2, 1.5, 3.4, 6.7, 7.2, 8.1]}$$

**دلیل درستی الگوریتم:**

چون بازه‌های مربوط به سطل‌ها مجزا و به ترتیب صعودی هستند، تمام عناصر یک سطل از عناصر سطل‌های بعدی کوچک‌ترند.

بنابراین اگر عناصر داخل هر سطل به درستی مرتب شوند، اتصال سطل‌ها به ترتیب شماره، یک آرایهٔ کاملاً مرتب‌شده ایجاد می‌کند.

**ب)**

**پیچیدگی زمانی در حالت متوسط:**

برای ساخت $n$ سطل و قرار دادن $n$ عدد در سطل‌های مربوط، در مجموع $O(n)$ زمان نیاز داریم. اتصال سطل‌ها نیز $O(n)$ زمان می‌برد.

اگر اعداد به صورت مستقل و یکنواخت در بازهٔ $[0, k)$ توزیع شده باشند، تعداد عناصر هر سطل به طور متوسط ثابت خواهد بود.

در این حالت، مجموع زمان مورد انتظار مرتب‌سازی تمام سطل‌ها با Insertion Sort برابر $O(n)$ خواهد بود.

بنابراین پیچیدگی زمانی مورد انتظار الگوریتم برابر است با:

$$\boxed{T_{\text{avg}}(n) = O(n)}$$

این نتیجه به فرض توزیع یکنواخت و مستقل داده‌ها وابسته است.

**پیچیدگی زمانی در بدترین حالت:**

در بدترین حالت ممکن است تمام $n$ عدد وارد یک سطل شوند و سایر سطل‌ها خالی بمانند.

در این شرایط باید یک سطل شامل $n$ عنصر را با Insertion Sort مرتب کنیم. چون پیچیدگی این روش در بدترین حالت $O(n^2)$ است، پیچیدگی کل Bucket Sort نیز برابر خواهد بود با:

$$\boxed{T_{\text{worst}}(n) = O(n^2)}$$

**ج)**

**شرایط ایجاد پیچیدگی درجهٔ دوم:**

اگر داده‌ها به جای توزیع یکنواخت، در یک بازهٔ کوچک متمرکز شده باشند، ممکن است تعداد زیادی از عناصر وارد یک سطل شوند.

اگر این عناصر در ترتیب نزولی قرار داشته باشند، مرتب‌سازی آن‌ها با Insertion Sort به تعداد زیادی مقایسه و جابه‌جایی نیاز خواهد داشت.

برای مثال، فرض می‌کنیم $n = 5$ و $k = 10$ باشد و آرایه به صورت زیر داده شود:

$$A = [1.9, 1.7, 1.5, 1.3, 1.1]$$

چون عرض هر سطل برابر 2 است، تمام این اعداد در بازهٔ $[0, 2)$ قرار می‌گیرند و وارد سطل شمارهٔ صفر می‌شوند.

بنابراین Insertion Sort باید تمام پنج عنصر را که به صورت نزولی قرار گرفته‌اند، مرتب کند.

در حالت کلی، اگر $n$ عنصر به همین شکل داخل یک سطل قرار بگیرند، تعداد جابه‌جایی‌ها برابر خواهد بود با:

$$1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}$$

در نتیجه زمان اجرا از مرتبهٔ $\Theta(n^2)$ خواهد بود.

**انتخاب روش بهتر:**

اگر توزیع داده‌ها مشخص نباشد و احتمال تجمع عناصر در یک سطل وجود داشته باشد، می‌توان به جای Bucket Sort از Merge Sort استفاده کرد.

برخلاف Bucket Sort با Insertion Sort، پیچیدگی زمانی Merge Sort در بدترین حالت نیز برابر است با:

$$\boxed{T(n) = O(n \log n)}$$

بنابراین وقتی دربارهٔ توزیع داده‌ها اطمینان نداریم و می‌خواهیم زمان اجرا در بدترین حالت نیز مناسب باشد، Merge Sort انتخاب مطمئن‌تری است.

**سؤال ۴**

**پاسخ:**

**الف)**

**ایدهٔ حل:**

در الگوریتم Quick Sort ابتدا یک عنصر به عنوان Pivot انتخاب می‌کنیم و با پیمایش آرایه، عناصر کوچک‌تر و بزرگ‌تر از آن را در دو بخش جدا قرار می‌دهیم. سپس همین عملیات را برای هر بخش به صورت بازگشتی انجام می‌دهیم.

برای تحلیل زمان اجرا باید دو موضوع را بررسی کنیم: هزینهٔ تقسیم آرایه در هر مرحله و تعداد سطوح بازگشت الگوریتم.

**محاسبهٔ هزینهٔ زمانی:**

در هر مرحله برای تقسیم یک آرایه با $n$ عضو، باید عناصر آن را نسبت به Pivot بررسی کنیم. بنابراین هزینهٔ تقسیم‌بندی برابر $O(n)$ است.

اگر Pivot تقریباً در وسط مقادیر آرایه قرار بگیرد، دو قسمت با اندازه‌های نزدیک به هم ایجاد می‌شوند. در این حالت اندازهٔ زیرمسئله‌ها در هر سطح تقریباً نصف می‌شود.

برای مثال، اگر آرایه 16 عنصر داشته باشد، تقسیم‌بندی‌های متعادل به شکل زیر خواهند بود:

$$16 \longrightarrow 8 \longrightarrow 4 \longrightarrow 2 \longrightarrow 1$$

در این حالت حدود $\log_2 n$ سطح بازگشت داریم. همچنین مجموع تعداد عناصری که در هر سطح پردازش می‌شوند، حداکثر از مرتبهٔ $O(n)$ است.

پس در حالت تقسیم‌بندی متعادل، زمان اجرا برابر است با:

$$T(n) = O(n) \times O(\log n)$$

$$\boxed{T(n) = O(n \log n)}$$

**بررسی آرایهٔ تصادفی و انتخاب Pivot اول:**

در صورت سؤال فرض شده که عناصر آرایه به صورت مستقل، تصادفی و یکنواخت توزیع شده‌اند و ترتیب مشخصی ندارند.

در چنین شرایطی، اولین عنصر لزوماً کوچک‌ترین یا بزرگ‌ترین عضو نیست. در واقع با فرض متمایز بودن عناصر، هر یک از $n$ رتبهٔ ممکن با احتمال $1/n$ می‌تواند متعلق به عنصر اول باشد.

بنابراین اگرچه همیشه اولین عنصر را انتخاب می‌کنیم، رتبهٔ آن در آرایه تصادفی است.

برای مثال، ممکن است Pivot در یک مرحله آرایه را به دو قسمت با اندازه‌های نزدیک تقسیم کند و در مرحله‌ای دیگر تقسیم‌بندی نامتعادل‌تری ایجاد شود.

نکته این است که در آرایهٔ تصادفی، تقسیم‌بندی نامتعادل در تمام مراحل به صورت همیشگی اتفاق نمی‌افتد. در تحلیل میانگین باید هزینهٔ تقسیم‌بندی‌ها را روی تمام رتبه‌های ممکن Pivot در نظر بگیریم.

از آنجا که انتخاب هر رتبه احتمال یکسانی دارد، می‌توان هزینهٔ متوسط هر مرحله را بر اساس تمام تقسیم‌بندی‌های ممکن محاسبه کرد. این تحلیل نشان می‌دهد که تعداد مقایسه‌های مورد انتظار در کل الگوریتم از مرتبهٔ $n \log n$ است.

در نتیجه با وجود ثابت بودن محل انتخاب Pivot، زمان اجرای مورد انتظار برابر است با:

$$\boxed{T_{\text{avg}}(n) = \Theta(n \log n)}$$

**محاسبهٔ زمان در بدترین حالت:**

حال فرض می‌کنیم آرایه از قبل به صورت صعودی مرتب شده باشد:

$$A = [1, 2, 3, 4, 5, 6]$$

اگر همیشه اولین عنصر را Pivot انتخاب کنیم، در هر مرحله کوچک‌ترین عنصر انتخاب می‌شود.
در نتیجه یک قسمت خالی و قسمت دیگر شامل تمام عناصر باقی‌مانده خواهد بود.
در مرحلهٔ اول حدود $n-1$ مقایسه، در مرحلهٔ دوم $n-2$ مقایسه و به همین ترتیب تا انتهای الگوریتم انجام می‌شود.
پس مجموع مقایسه‌ها برابر است با:

$$(n-1) + (n-2) + \cdots + 2 + 1$$

با استفاده از مجموع اعداد متوالی داریم:

$$T(n) \propto \frac{n(n-1)}{2}$$

بنابراین:

$$\boxed{T_{\text{worst}}(n) = \Theta(n^2)}$$

پس انتخاب اولین عنصر به عنوان Pivot به تنهایی باعث نمی‌شود الگوریتم همیشه در بدترین حالت اجرا شود. برای آرایهٔ تصادفی، پیچیدگی مورد انتظار همچنان $\Theta(n \log n)$ است؛ اما در ورودی‌هایی مانند آرایهٔ مرتب‌شده، زمان اجرا می‌تواند به $\Theta(n^2)$ برسد.

**ب)**

**ایدهٔ حل:**

برای ساخت یک Max-Heap دو روش را با هم مقایسه می‌کنیم.
در روش استاندارد BUILD-MAX-HEAP، تمام عناصر از ابتدا در آرایه قرار دارند و با اجرای MAX-HEAPIFY از آخرین گره داخلی به سمت ریشه، خاصیت هیپ برقرار می‌شود.
اما در روش دوم، از یک هیپ خالی شروع می‌کنیم و عناصر را یکی‌یکی با INSERT وارد می‌کنیم.

**روش اول: ساخت Heap با درج متوالی**

در عملیات INSERT ابتدا عنصر جدید در انتهای هیپ قرار می‌گیرد. سپس مقدار آن با والدش مقایسه می‌شود و اگر بزرگ‌تر باشد، با والد جابه‌جا می‌شود.
این عملیات تا زمانی ادامه پیدا می‌کند که عنصر به جای مناسب برسد یا ریشه شود.
برای مثال، آرایهٔ زیر را در نظر می‌گیریم:

$$A = [4, 10, 3, 5, 8]$$

ابتدا با درج سه عنصر اول، هیپ زیر ساخته می‌شود:

$$H = [10, 4, 3]$$

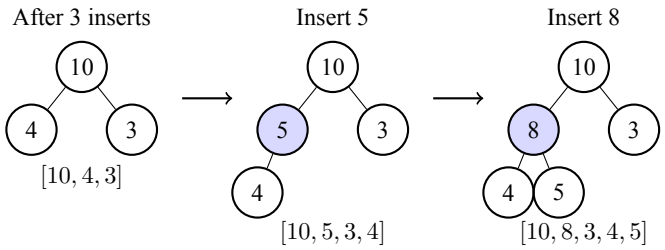
وقتی عدد $5$ را درج می‌کنیم، ابتدا در انتهای هیپ قرار می‌گیرد و چون از والدش یعنی $4$ بزرگ‌تر است، با آن جابه‌جا می‌شود.
پس از این مرحله داریم:

$$H = [10, 5, 3, 4]$$

درج عدد $8$ نیز باعث جابه‌جایی آن با والدش یعنی $5$ می‌شود و هیپ نهایی به شکل زیر خواهد بود:

$$H = [10, 8, 3, 4, 5]$$

شکل زیر وضعیت هیپ را پس از این درج‌ها نشان می‌دهد.




**محاسبهٔ پیچیدگی روش :INSERT**

هیپ یک درخت دودویی کامل است. در چنین درختی، ارتفاع برای $n$ عنصر از مرتبهٔ $\log n$ است.

در بدترین حالت، عنصر جدید باید از انتهای هیپ تا ریشه حرکت کند. بنابراین یک عملیات INSERT حداکثر $O(\log n)$ زمان می‌برد.

حال فرض می‌کنیم $n$ عنصر را به ترتیب وارد هیپ کنیم.

درج عنصر اول هزینهٔ ثابتی دارد. درج عناصر بعدی ممکن است به تعداد سطوح هیپ نیاز به مقایسه و جابه‌جایی داشته باشد.

پس برای درج $n$ عنصر، می‌توانیم هزینهٔ کل را به صورت زیر بنویسیم:

$$T(n) = O(\log 1) + O(\log 2) + \cdots + O(\log n)$$

چون هر جمله حداکثر $O(\log n)$ است و $n$ عنصر داریم:

$$T(n) = O(n \log n)$$

این کران در بدترین حالت نیز قابل دستیابی است. برای مثال، اگر اعداد به ترتیب صعودی وارد شوند، هر عنصر جدید از عناصر قبلی بزرگ‌تر است و باید تا ریشه حرکت کند.

برای تعداد زیادی از درج‌ها، این حرکت در حدود $\log n$ سطح خواهد بود. بنابراین در بدترین حالت:

$$\boxed{T_{\text{INSERT}}(n) = \Theta(n \log n)}$$

**روش دوم: ساخت استاندارد BUILD-MAX-HEAP**

در این روش ابتدا تمام $n$ عنصر را در آرایه قرار می‌دهیم.

سپس از آخرین گره داخلی شروع کرده و با اجرای MAX-HEAPIFY روی گره‌ها، خاصیت Max-Heap را از پایین به بالا برقرار می‌کنیم.

در نگاه اول ممکن است تصور کنیم چون هر MAX-HEAPIFY حداکثر $O(\log n)$ زمان دارد، زمان کل نیز $O(n \log n)$ خواهد بود.

اما این تحلیل دقیق نیست؛ زیرا تمام گره‌ها به یک اندازه نیاز به جابه‌جایی ندارند.

**محاسبهٔ هزینهٔ روش استاندارد:**

در یک هیپ با $n$ عنصر، تقریباً نصف گره‌ها برگ هستند. این گره‌ها نیازی به MAX-HEAPIFY ندارند.
حدود یک‌چهارم گره‌ها در ارتفاع یک قرار دارند و حداکثر یک سطح ممکن است به پایین حرکت کنند.
حدود یک‌هشتم گره‌ها در ارتفاع دو هستند و حداکثر دو سطح حرکت می‌کنند.
به همین ترتیب، هرچه ارتفاع بیشتر می‌شود، تعداد گره‌های آن ارتفاع کمتر خواهد شد.
بنابراین می‌توان هزینه‌ها را به صورت زیر در نظر گرفت:

| Height | Approx. nodes | Max. work per node |
|---|---|---|
| 0 | $n/2$ | 0 |
| 1 | $n/4$ | 1 |
| 2 | $n/8$ | 2 |
| 3 | $n/16$ | 3 |
| ⋮ | ⋮ | ⋮ |

برای محاسبهٔ کار کل، تعداد گره‌های هر ارتفاع را در حداکثر تعداد سطوحی که ممکن است حرکت کنند، ضرب کرده و نتیجه‌ها را جمع می‌کنیم.
پس هزینهٔ کل متناسب است با:

$$T(n) \propto \frac{n}{4} + 2\frac{n}{8} + 3\frac{n}{16} + \cdots$$

اگر $n$ را فاکتور بگیریم:

$$T(n) \propto n\left(\frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \cdots\right)$$

عبارت داخل پرانتز یک مجموع همگراست و با افزایش $n$ از یک مقدار ثابت بیشتر نمی‌شود.
در نتیجه، مجموع کار تمام گره‌ها از مرتبهٔ $O(n)$ خواهد بود. از طرفی، ساخت و پردازش اولیهٔ آرایه نیز حداقل زمان خطی نیاز دارد.
پس پیچیدگی زمانی روش استاندارد برابر است با:

$$\boxed{T_{\text{BUILD-MAX-HEAP}}(n) = \Theta(n)}$$

**مقایسهٔ نهایی دو روش:**
در روش درج متوالی، هر عنصر ممکن است تا ریشه حرکت کند و این عملیات برای تمام عناصر تکرار می‌شود.
اما در روش استاندارد، بیشتر گره‌ها نزدیک برگ‌ها قرار دارند و هزینهٔ اصلاح آن‌ها بسیار کم است. به همین دلیل مجموع عملیات خطی می‌شود.

| Method | Worst-case time |
|---|---|
| Repeated INSERT | $\Theta(n \log n)$ |
| BUILD-MAX-HEAP | $\Theta(n)$ |

بنابراین روش درج متوالی در بدترین حالت از روش استاندارد کندتر است و برای ساخت اولیهٔ یک هیپ از آرایهٔ موجود، استفاده از BUILD-MAX-HEAP انتخاب مناسب‌تری خواهد بود.

**سؤال ۵**

**پاسخ:**

**ایدهٔ حل:**

در این سؤال هدف پیدا کردن $k$امین عنصر بزرگ‌تر آرایه است، بدون اینکه تمام عناصر را مرتب کنیم.

برای این کار می‌توانیم از الگوریتم QuickSelect استفاده کنیم. این الگوریتم مشابه QuickSort ابتدا یک عنصر را به عنوان Pivot انتخاب کرده و آرایه را نسبت به آن تقسیم می‌کند. با این تفاوت که فقط در بخشی که عنصر مورد نظر قرار دارد، جست‌وجو را ادامه می‌دهد.

اگر Pivot نامناسب انتخاب شود، ممکن است در هر مرحله فقط یک عنصر حذف شود و زمان اجرا در بدترین حالت به $O(n^2)$ برسد.

برای جلوگیری از این مشکل، از روش Median of Medians استفاده می‌کنیم که یک Pivot با کیفیت تضمین‌شده انتخاب می‌کند و باعث می‌شود زمان اجرای الگوریتم در بدترین حالت نیز خطی باقی بماند.

**الف)**

**مراحل الگوریتم Median of Medians:**

ابتدا آرایه را به گروه‌های پنج‌تایی تقسیم می‌کنیم. اگر تعداد عناصر مضربی از پنج نباشد، گروه آخر می‌تواند کمتر از پنج عضو داشته باشد.

سپس مراحل زیر را انجام می‌دهیم:

۱. هر گروه را به صورت جداگانه مرتب کرده و میانهٔ آن را پیدا می‌کنیم. چون هر گروه حداکثر پنج عضو دارد، مرتب‌سازی هر گروه زمان ثابت نیاز دارد.

۲. میانه‌های به دست آمده را در یک آرایهٔ جدید قرار می‌دهیم. تعداد این میانه‌ها تقریباً $n/5$ خواهد بود.

۳. میانهٔ آرایهٔ جدید را با همین روش انتخاب می‌کنیم و آن را به عنوان Pivot آرایهٔ اصلی در نظر می‌گیریم.

۴. آرایهٔ اصلی را نسبت به Pivot به سه بخش کوچک‌تر، مساوی و بزرگ‌تر تقسیم می‌کنیم.

۵. با توجه به تعداد عناصر هر بخش و مقدار $k$، یا پاسخ را پیدا می‌کنیم یا فقط در بخش مورد نیاز جست‌وجو را ادامه می‌دهیم.

**بررسی مثال:**

فرض می‌کنیم آرایهٔ زیر شامل $25$ عنصر باشد و بخواهیم پنجمین عدد بزرگ‌تر آن را پیدا کنیم.

آرایه را به پنج گروه تقسیم کرده و میانهٔ هر گروه را محاسبه می‌کنیم.

| Group | Elements | Median |
|---|---|---|
| 1 | 9, 1, 5, 3, 7 | 5 |
| 2 | 10, 6, 8, 2, 4 | 6 |
| 3 | 15, 11, 14, 12, 13 | 13 |
| 4 | 20, 16, 19, 17, 18 | 18 |
| 5 | 25, 21, 24, 22, 23 | 23 |

آرایهٔ میانه‌ها برابر است با:

$$M = [5, 6, 13, 18, 23]$$

میانهٔ این آرایه عدد $13$ است. بنابراین آن را به عنوان Pivot انتخاب می‌کنیم.

پس از تقسیم آرایه نسبت به عدد 13، تعداد عناصر هر بخش به صورت زیر است:

$$\underbrace{12}_{\text{کوچک‌تر}} \quad | \quad \underbrace{13}_{\text{Pivot}} \quad | \quad \underbrace{12}_{\text{بزرگ‌تر}}$$

چون 12 عدد از Pivot بزرگ‌تر هستند و ما به دنبال پنجمین عدد بزرگ‌تر هستیم، پاسخ در بخش بزرگ‌تر قرار دارد.
بنابراین فقط همین بخش را بررسی می‌کنیم:

$$[14, 15, 16, \ldots, 25]$$

با ادامهٔ الگوریتم، پنجمین عدد بزرگ‌تر برابر 21 به دست می‌آید.

$$\boxed{\text{Answer} = 21}$$

**دلیل انتخاب گروه‌های پنج‌تایی:**
مهم‌ترین ویژگی این روش آن است که Pivot انتخاب‌شده نمی‌تواند بیش از حد به ابتدا یا انتهای ترتیب مرتب‌شده نزدیک باشد.
برای بررسی این موضوع، یک گروه پنج‌تایی مرتب‌شده را در نظر می‌گیریم:

$$[2, 5, 8, 11, 14]$$

میانهٔ این گروه عدد 8 است. بنابراین حداقل سه عدد این گروه بزرگ‌تر یا مساوی میانه هستند و حداقل سه عدد نیز کوچک‌تر یا مساوی آن هستند.
حالا فرض می‌کنیم $p$ میانهٔ میانه‌های تمام گروه‌ها باشد.
چون $p$ میانهٔ آرایهٔ میانه‌هاست، حداقل نصف میانه‌های گروه‌ها بزرگ‌تر یا مساوی $p$ هستند.
در هر کدام از این گروه‌ها نیز حداقل سه عنصر بزرگ‌تر یا مساوی میانهٔ همان گروه وجود دارد. پس این سه عنصر از $p$ نیز بزرگ‌تر یا مساوی هستند.
تعداد گروه‌ها تقریباً $n/5$ است. بنابراین تعداد عناصری که به این روش تضمین می‌شوند، تقریباً برابر است با:

$$\frac{n}{5} \times \frac{1}{2} \times 3 = \frac{3n}{10}$$

به همین صورت، حداقل حدود $3n/10$ عنصر نیز کوچک‌تر یا مساوی $p$ هستند.
با در نظر گرفتن گروه ناقص احتمالی و چند عنصر مرزی، تعداد عناصر تضمین‌شده در هر سمت برابر $3n/10 - O(1)$ خواهد بود.
بنابراین بعد از تقسیم‌بندی، در بدترین حالت اندازهٔ بخش باقی‌مانده حداکثر حدود زیر است:

$$n - \frac{3n}{10} = \frac{7n}{10}$$

این یعنی در هر مرحله حداقل حدود 30 درصد عناصر از جست‌وجوی بعدی حذف می‌شوند.

**چرا پنج عضو انتخاب می‌کنیم؟**
اگر گروه‌ها سه‌تایی باشند، از هر گروه فقط دو عنصر را در یک سمت میانه تضمین می‌کنیم.
در این حالت تعداد عناصر تضمین‌شده تقریباً برابر است با:

$$\frac{n}{3} \times \frac{1}{2} \times 2 = \frac{n}{3}$$

پس اندازهٔ بخش باقی‌مانده می‌تواند تا حدود $2n/3$ باشد.
از طرفی برای انتخاب میانهٔ میانه‌ها نیز باید یک زیرمسئلهٔ $n/3$ عضوی را حل کنیم.
در نتیجه مجموع اندازهٔ دو زیرمسئله تقریباً برابر $n$ می‌شود و این تحلیل، زمان خطی را تضمین نمی‌کند.
اما با گروه‌های پنج‌تایی، اندازهٔ دو زیرمسئله به ترتیب حدود $n/5$ و $7n/10$ است که مجموع آن‌ها از $n$ کمتر است.
بنابراین گروه‌های پنج‌تایی یک انتخاب کوچک و مناسب برای تضمین زمان خطی الگوریتم هستند.

**ب)**

**محاسبهٔ پیچیدگی زمانی در بدترین حالت:**

برای محاسبهٔ زمان اجرای الگوریتم، هزینهٔ هر مرحله را جداگانه بررسی می‌کنیم.

**مرحلهٔ اول: تقسیم و پیدا کردن میانه‌ها**

آرایه را به تقریباً $n/5$ گروه تقسیم می‌کنیم.
مرتب کردن هر گروه پنج‌تایی و پیدا کردن میانهٔ آن زمان ثابت نیاز دارد.
پس برای تمام گروه‌ها داریم:

$$\frac{n}{5} \times O(1) = O(n)$$

**مرحلهٔ دوم: انتخاب Pivot**

برای پیدا کردن میانهٔ میانه‌ها، باید یک مسئلهٔ انتخاب روی آرایه‌ای با تقریباً $n/5$ عنصر انجام دهیم.
چون همین روش به صورت بازگشتی استفاده می‌شود، هزینهٔ این مرحله برابر است با:

$$T(n/5)$$

**مرحلهٔ سوم: تقسیم‌بندی آرایه**

بعد از انتخاب Pivot، تمام عناصر آرایه را نسبت به آن بررسی و تقسیم‌بندی می‌کنیم.
چون هر عنصر یک بار بررسی می‌شود، هزینهٔ این مرحله برابر است با:

$$O(n)$$

**مرحلهٔ چهارم: ادامهٔ جست‌وجو**

طبق نتیجهٔ قسمت الف، در بدترین حالت حداکثر حدود $7n/10$ عنصر برای ادامهٔ جست‌وجو باقی می‌ماند.
بنابراین هزینهٔ این قسمت برابر است با:

$$T(7n/10)$$

**تشکیل رابطهٔ بازگشتی:**

با جمع کردن هزینهٔ مراحل قبل، و صرف‌نظر از ثابت‌های مربوط به گروه ناقص، داریم:

$$T(n) \leq T(n/5) + T(7n/10) + cn$$

در این رابطه، $cn$ هزینهٔ گروه‌بندی، پیدا کردن میانه‌های پنج‌تایی و تقسیم‌بندی آرایه است.
حالا مجموع اندازهٔ دو زیرمسئله را حساب می‌کنیم:

$$\frac{n}{5} + \frac{7n}{10} = \frac{2n}{10} + \frac{7n}{10} = \frac{9n}{10}$$

بنابراین مجموع اندازهٔ زیرمسئله‌های بازگشتی، حداکثر حدود 90 درصد اندازهٔ مسئلهٔ اولیه است.
اگر هزینهٔ پردازش سطح اول را $cn$ در نظر بگیریم، مجموع هزینهٔ سطح بعدی حداکثر متناسب با $0.9cn$ خواهد بود.
در سطوح بعدی نیز همین کاهش ادامه پیدا می‌کند.
بنابراین مجموع هزینه‌ها با یک دنبالهٔ هندسی کراندار می‌شود:

$$T(n) \lesssim cn + 0.9cn + (0.9)^2 cn + \cdots$$

مجموع این دنباله برابر است با:

$$cn\left(1 + 0.9 + 0.9^2 + \cdots\right) = \frac{cn}{1 - 0.9}$$

پس:

$$T(n) \lesssim 10cn$$

در نتیجه زمان اجرای الگوریتم از مرتبهٔ $O(n)$ است.
از طرفی برای گروه‌بندی و تقسیم‌بندی اولیه، حداقل زمان خطی نیاز داریم. بنابراین پیچیدگی در بدترین حالت برابر می‌شود با:

$$\boxed{T_{\text{worst}}(n) = \Theta(n)}$$

این نتیجه نشان می‌دهد که با انتخاب Pivot به روش Median of Medians، حتی در بدترین حالت نیز زمان اجرای الگوریتم خطی است.

## ج)

### مقایسه با QuickSelect تصادفی:

در الگوریتم QuickSelect معمولی، یک Pivot انتخاب کرده و آرایه را نسبت به آن تقسیم می‌کنیم.
سپس فقط در بخشی که عنصر مورد نظر قرار دارد ادامه می‌دهیم.
اگر Pivot به صورت تصادفی انتخاب شود، در حالت متوسط تعداد مناسبی از عناصر در هر مرحله حذف می‌شوند.
بنابراین زمان اجرای مورد انتظار QuickSelect تصادفی برابر است با:

$$\boxed{E[T(n)] = \Theta(n)}$$

اما این روش تضمینی برای مناسب بودن Pivot در تمام مراحل ندارد.
در بدترین حالت ممکن است هر بار کوچک‌ترین یا بزرگ‌ترین عنصر انتخاب شود.
در این صورت اندازهٔ زیرمسئله‌ها به ترتیب $n-1$، $n-2$ و ... خواهد بود.
پس مجموع هزینهٔ مراحل برابر است با:

$$n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2}$$

و در نتیجه:

$$\boxed{T_{\text{worst}}(n) = \Theta(n^2)}$$

در مقابل، روش Median of Medians با انتخاب یک Pivot با کیفیت تضمین‌شده، در بدترین حالت نیز زمان خطی دارد.
مقایسهٔ دو روش به صورت زیر است:

| Algorithm | Expected Time | Worst Case |
|---|---|---|
| Randomized QuickSelect | $\Theta(n)$ | $\Theta(n^2)$ |
| Median of Medians | $\Theta(n)$ | $\Theta(n)$ |

**دلیل استفادهٔ بیشتر از QuickSelect در عمل:**

با وجود تضمین زمانی بهتر در Median of Medians، معمولاً QuickSelect تصادفی در عمل انتخاب رایج‌تری است.
دلیل این موضوع آن است که در QuickSelect تصادفی، انتخاب Pivot بسیار ساده و با هزینهٔ ثابت انجام می‌شود.
اما در Median of Medians، برای انتخاب Pivot باید آرایه را گروه‌بندی کنیم، میانهٔ گروه‌ها را پیدا کنیم و یک انتخاب بازگشتی دیگر نیز انجام دهیم.
این عملیات باعث افزایش هزینهٔ ثابت الگوریتم می‌شوند.

بنابراین اگرچه هر دو الگوریتم زمان خطی دارند، QuickSelect تصادفی معمولاً در ورودی‌های معمولی سریع‌تر اجرا می‌شود.
در نتیجه زمانی که تضمین زمان اجرا در بدترین حالت اهمیت زیادی داشته باشد، Median of Medians انتخاب مناسب‌تری است.
اما برای کاربردهای معمولی، به دلیل سادگی پیاده‌سازی و هزینهٔ ثابت کمتر، اغلب از QuickSelect تصادفی استفاده می‌شود.

## پاسخ سؤالات عملی

---

### سؤال ۶: نبرد هوایی

**ایدهٔ حل:** در این سؤال، ارتفاع جنگنده‌ها به ترتیب قرار گرفتن آن‌ها در یک آرایه داده شده است. هر جنگنده می‌تواند جنگنده‌های بعد از خودش را هدف قرار دهد، به شرطی که ارتفاع کمتری داشته باشند. بنابراین برای هر دو اندیس $i$ و $j$، اگر $i < j$ و $h_i > h_j$ باشد، یک هدف معتبر داریم. در واقع پاسخ مسئله برابر تعداد وارونگی‌های آرایه است.

در ابتدا می‌توان برای هر جنگنده، همهٔ جنگنده‌های بعد از آن را بررسی کرد و در صورت کمتر بودن ارتفاعشان، یک واحد به پاسخ اضافه کرد. اما این روش به دو حلقهٔ تودرتو نیاز دارد و پیچیدگی آن $O(n^2)$ است. با توجه به اینکه تعداد جنگنده‌ها می‌تواند تا $10^5$ باشد و محدودیت زمانی مسئله یک ثانیه است، از روش شمارش وارونگی با Merge Sort استفاده کردم.

**مراحل الگوریتم:** آرایه را از وسط به دو نیمه تقسیم می‌کنیم و برای هر نیمه، همین کار را به صورت بازگشتی ادامه می‌دهیم. وقتی اندازهٔ یک بخش به یک رسید، آن بخش وارونگی ندارد. سپس هنگام ادغام دو نیمهٔ مرتب‌شده، علاوه بر مرتب کردن عناصر، وارونگی‌هایی را که یک عضو آن‌ها در نیمهٔ چپ و عضو دیگر در نیمهٔ راست قرار دارد، می‌شماریم.

برای ادغام، دو اشاره‌گر $i$ و $j$ به ابتدای نیمه‌های چپ و راست اشاره می‌کنند. اگر $h_i \leq h_j$ باشد، عنصر سمت چپ را در آرایهٔ موقت قرار می‌دهیم؛ چون در این حالت وارونگی جدیدی بین عنصر فعلی چپ و عناصر باقی‌ماندهٔ راست به دست نمی‌آید. اگر $h_i > h_j$ باشد، عنصر سمت راست را وارد آرایهٔ موقت می‌کنیم. چون نیمهٔ چپ از قبل مرتب شده است، همهٔ عناصر باقی‌ماندهٔ آن نیمه نیز از $h_j$ بزرگ‌تر هستند. پس تعداد وارونگی‌های جدید برابر است با:

$$\text{mid} - i + 1$$

این مقدار را به شمارنده اضافه می‌کنیم و ادغام را ادامه می‌دهیم. در پایان نیز عناصر باقی‌مانده را کپی می‌کنیم و آرایهٔ موقت را به آرایهٔ اصلی برمی‌گردانیم.

نکته این است که مرتب کردن هر نیمه مشکلی برای ترتیب اولیه ایجاد نمی‌کند؛ چون تمام عناصر نیمهٔ چپ، در آرایهٔ اولیه قبل از عناصر نیمهٔ راست بوده‌اند. همچنین وارونگی‌های داخل هر نیمه در فراخوانی‌های بازگشتی محاسبه می‌شوند. به این ترتیب هر وارونگی دقیقاً یک بار شمرده می‌شود.

**بررسی مثال:** برای ورودی نمونهٔ اول، آرایهٔ ارتفاع‌ها برابر $[5, 4, 3, 7, 6]$ است. جفت‌های وارونه عبارت‌اند از $(5, 4)$، $(5, 3)$، $(4, 3)$ و $(7, 6)$. بنابراین پاسخ نهایی 4 است که با خروجی نمونه مطابقت دارد.

**نحوهٔ پیاده‌سازی:** در کد دو تابع اصلی در نظر گرفتم. تابع mergeSort ابتدا دو نیمهٔ آرایه را به صورت بازگشتی پردازش می‌کند و سپس تابع merge را فراخوانی می‌کند. تابع merge با استفاده از آرایهٔ موقت temp دو نیمه را ادغام کرده و تعداد وارونگی‌های بین آن‌ها را برمی‌گرداند. جواب هر فراخوانی برابر مجموع وارونگی‌های نیمهٔ چپ، نیمهٔ راست و وارونگی‌های بین دو نیمه است.

برای متغیر شمارنده و مقدار بازگشتی توابع از نوع long long استفاده کردم؛ چون در بدترین حالت، وقتی ارتفاع‌ها به ترتیب نزولی باشند، تعداد وارونگی‌ها برابر $\frac{n(n-1)}{2}$ است که برای $n = 100000$ از محدودهٔ int بیشتر می‌شود.

**پیچیدگی زمانی و حافظه:** در هر مرحله، آرایه به دو نیمه تقسیم می‌شود و ادغام عناصر هر سطح در مجموع $O(n)$ زمان می‌برد. بنابراین رابطهٔ بازگشتی زمان اجرا برابر است با

$$T(n) = 2T(n/2) + O(n) = O(n \log n).$$

همچنین آرایهٔ موقت برای ادغام به $O(n)$ حافظه نیاز دارد و عمق پشتهٔ بازگشت $O(\log n)$ است؛ در نتیجه پیچیدگی حافظهٔ کل $O(n)$ خواهد بود.

### سؤال ۷: کیک تکه‌تکه

**ایدهٔ حل:** در این سؤال، اندازهٔ قسمت‌های دو ضلع کیک در آرایه‌های arz و tol داده شده است. اگر تعداد اعضای این آرایه‌ها به ترتیب $n$ و $m$ باشد، کیک در مجموع $n \times m$ تکه دارد و مساحت هر تکه از ضرب یک عضو آرایهٔ اول در یک عضو آرایهٔ دوم به دست می‌آید. چون منیژه ابتدا $k-1$ تکهٔ بزرگ‌تر را به دوستانش می‌دهد، باید مساحت $k$امین تکهٔ بزرگ را پیدا کنیم.

در ابتدا می‌توان همهٔ $n \times m$ حاصل‌ضرب را محاسبه و به صورت نزولی مرتب کرد. اما تعداد تکه‌ها ممکن است تا $10^{10}$ برسد و ساختن همهٔ آن‌ها از نظر زمان و حافظه مناسب نیست. به همین دلیل از Merge Sort برای مرتب کردن دو آرایه، روش دو اشاره‌گر برای شمارش تکه‌ها و جست‌وجوی دودویی روی مساحت جواب استفاده کردم.

**مراحل الگوریتم:** ابتدا دو آرایه را به صورت صعودی با Merge Sort مرتب می‌کنیم. سپس برای یک مساحت فرضی $x$، تعداد جفت‌هایی را می‌شماریم که شرط زیر را داشته باشند:

$$\text{arz}[i] \times \text{tol}[j] \geq x$$

برای این کار، اشاره‌گر $i$ را از ابتدای آرایهٔ arz و اشاره‌گر $j$ را از انتهای آرایهٔ tol شروع می‌کنیم. برای هر $i$، تا وقتی حاصل‌ضرب عنصر فعلی در $\text{tol}[j]$ حداقل $x$ است، $j$ را یک خانه به چپ می‌بریم. چون آرایهٔ tol مرتب است، پس از توقف، همهٔ $m-j-1$ عنصر سمت راست $j$ حاصل‌ضرب قابل قبولی دارند و این تعداد را به شمارنده اضافه می‌کنیم.

با بزرگ‌تر شدن $i$، مقدار arz[i] کاهش پیدا نمی‌کند؛ بنابراین $j$ فقط به چپ حرکت می‌کند و لازم نیست برای هر عضو از ابتدا بررسی شود. در نتیجه شمارش برای هر مقدار $x$ در زمان $O(n+m)$ انجام می‌شود.

حالا روی مساحت جست‌وجوی دودویی انجام می‌دهیم. بازهٔ اولیه از 1 تا حاصل‌ضرب بزرگ‌ترین اعضای دو آرایه است. در هر مرحله مقدار $mid$ را حساب می‌کنیم. اگر تعداد تکه‌هایی با مساحت حداقل $mid$، بزرگ‌تر یا مساوی $k$ باشد، $mid$ را در ans ذخیره می‌کنیم و نیمهٔ بزرگ‌تر بازه را بررسی می‌کنیم. در غیر این صورت، سراغ نیمهٔ کوچک‌تر می‌رویم. این روند تا زمانی که کران پایین از کران بالا بیشتر شود ادامه دارد. در پایان، ans بزرگ‌ترین مساحتی است که دست‌کم $k$ تکه با آن مساحت یا بیشتر وجود دارد؛ پس همان جواب مسئله است.

**بررسی مثال:** فرض می‌کنیم $\text{arz} = [2, 4]$، $\text{tol} = [3, 5, 7]$ و $k = 3$ باشد. مساحت تکه‌ها برابر 6، 10، 14، 12، 20 و 28 است. ترتیب نزولی آن‌ها $[28, 20, 14, 12, 10, 6]$ می‌شود و پاسخ باید 14 باشد. در جست‌وجوی دودویی، برای $mid = 14$ دقیقاً سه تکه با مساحت حداقل 14 داریم؛ اما برای هر مقدار بزرگ‌تر از 14 فقط دو تکه یا کمتر وجود دارد. پس الگوریتم مقدار 14 را برمی‌گرداند.

**نحوهٔ پیاده‌سازی:** در کد، دو تابع mergeSort و merge مرتب‌سازی صعودی دو آرایه را انجام می‌دهند. تابع countPieces با استفاده از دو اشاره‌گر، تعداد تکه‌هایی با مساحت حداقل $x$ را محاسبه می‌کند. تابع solve ابتدا آرایه‌ها را مرتب می‌کند و سپس با جست‌وجوی دودویی و فراخوانی countPieces، مساحت مورد نظر را به دست می‌آورد. هر سناریو نیز به طور مستقل در تابع main خوانده و پاسخ آن چاپ می‌شود.

برای نگهداری اندازه‌ها، مساحت‌ها، مقدار $k$ و شمارنده از نوع long long استفاده کردم؛ چون مساحت یک تکه ممکن است به $10^{18}$ و تعداد کل تکه‌ها به $10^{10}$ برسد و این مقادیر در محدودهٔ int قرار نمی‌گیرند.

**پیچیدگی زمانی و حافظه:** مرتب کردن دو آرایه با Merge Sort به $O(n \log n + m \log m)$ زمان نیاز دارد. هر بار اجرای تابع شمارش $O(n+m)$ زمان می‌برد و چون بیشترین مساحت ممکن $10^{18}$ است، جست‌وجوی دودویی حداکثر حدود 60 مرحله دارد. بنابراین پیچیدگی زمانی کل برابر است با:

$$O\big(n \log n + m \log m + (n+m) \log(10^{18})\big).$$

همچنین برای نگهداری دو آرایه و فضای موقت مرتب‌سازی به $O(n+m)$ حافظه نیاز داریم.

**سؤال ۸: عدالت شکلاتی**

**ایدهٔ حل:**

در این سؤال یک کیک مکعب‌مستطیلی داریم که تعدادی قطعه شکلات در قسمت‌های مختلف آن قرار گرفته‌اند. هدف این است که کیک را با $m-1$ برش موازی صفحهٔ $yz$ به $m$ قسمت تقسیم کنیم، به طوری که حجم شکلات موجود در هر قسمت یکسان باشد. بنابراین باید مختصات برش‌ها را روی محور $x$ پیدا کنیم.

در ابتدا می‌توان برای هر برش، حجم شکلات موجود در سمت چپ یک مختصات مشخص را محاسبه کرد و محل مناسب را با جست‌وجوی دودویی به دست آورد. اما با توجه به اینکه طول کیک در راستای محور $x$ حداکثر $100$ است و مختصات قطعات شکلات نیز صحیح هستند، می‌توان از روش ساده‌تری استفاده کرد.

برای این کار، کیک را در راستای محور $x$ به لایه‌های یک‌واحدی تقسیم می‌کنیم و حجم شکلات موجود در هر لایه را در یک آرایه ذخیره می‌کنیم. سپس با محاسبهٔ حجم تجمعی شکلات از ابتدای کیک، مختصات برش‌ها را به دست می‌آوریم.

**مراحل الگوریتم:**

ابتدا آرایه‌ای به نام box با طول $a$ و مقدار اولیهٔ صفر می‌سازیم. عنصر box[x] نشان‌دهندهٔ حجم شکلات موجود در بازهٔ $[x, x+1]$ از کیک است.

هر قطعه شکلات با مختصات $(x_1, y_1, z_1)$ و $(x_2, y_2, z_2)$ مشخص می‌شود. مساحت مقطع این قطعه در صفحهٔ $yz$ برابر است با:

$$S = (y_2 - y_1)(z_2 - z_1)$$

از آنجا که قطعات شکلات مکعب‌مستطیلی هستند، مساحت مقطع آن‌ها در راستای محور $x$ ثابت است. بنابراین مقدار $S$ را به تمام عناصر آرایهٔ box از اندیس $x_1$ تا $x_2 - 1$ اضافه می‌کنیم.

همچنین حجم هر قطعه شکلات برابر است با:

$$V_i = S(x_2 - x_1)$$

با جمع کردن حجم تمام قطعات، حجم کل شکلات را در متغیر hajm به دست می‌آوریم.

پس از محاسبهٔ حجم کل، باید محل $m-1$ برش را پیدا کنیم. چون قرار است حجم شکلات هر قسمت برابر باشد، حجم تجمعی مورد نیاز تا برش شمارهٔ $i$ برابر است با:

$$target_i = \frac{i \times hajm}{m}$$

برای پیدا کردن هر برش، از یک اشاره‌گر $j$ استفاده می‌کنیم که شمارهٔ لایهٔ فعلی را نگه می‌دارد. متغیر sum نیز حجم شکلات تمام لایه‌هایی را که به طور کامل از آن‌ها عبور کرده‌ایم، ذخیره می‌کند.

تا زمانی که مجموع sum و box[j] از مقدار هدف کمتر باشد، کل لایهٔ فعلی را در نظر گرفته و اشاره‌گر را یک خانه جلو می‌بریم.

وقتی به لایه‌ای برسیم که حجم مورد نظر در آن قرار دارد، باید مشخص کنیم چه کسری از آن لایه برای رسیدن به حجم هدف لازم است.

حجم شکلات مورد نیاز در لایهٔ فعلی برابر است با:

$$target_i - sum$$

از طرفی، حجم شکلات در هر واحد طول این لایه برابر box[j] است. بنابراین مختصات برش از رابطهٔ زیر به دست می‌آید:

$$\boxed{X_i = j + \frac{target_i - sum}{box[j]}}$$

اگر مقدار حجم هدف دقیقاً در مرز یک لایه به دست آید، برش می‌تواند در همان مرز قرار بگیرد.

از آنجا که حجم تجمعی مورد نیاز برای برش‌ها به ترتیب افزایش پیدا می‌کند، اشاره‌گر $j$ فقط به سمت جلو حرکت می‌کند و نیازی نیست برای هر برش پیمایش را از ابتدای کیک شروع کنیم.

**بررسی مثال:**

برای ورودی نمونهٔ سؤال، دو قطعه شکلات در یک کیک با ابعاد $3 \times 4 \times 5$ قرار گرفته‌اند و تعداد افراد برابر $3$ است.

حجم قطعهٔ اول برابر است با:

$$V_1 = (1-0)(1-0)(1-0) = 1$$

حجم قطعهٔ دوم نیز برابر است با:

$$V_2 = (3-2)(4-2)(5-2) = 6$$

بنابراین حجم کل شکلات برابر $7$ است و سهم هر نفر برابر خواهد بود با:

$$\frac{7}{3}$$

آرایهٔ حجم شکلات در لایه‌های محور $x$ به صورت زیر تشکیل می‌شود:

$$box = [1, 0, 6]$$

برای برش اول باید حجم تجمعی شکلات به $\frac{7}{3}$ برسد. تا ابتدای لایهٔ سوم، یک واحد شکلات داریم و در این لایه شش واحد شکلات وجود دارد. بنابراین:

$$X_1 = 2 + \frac{\frac{7}{3} - 1}{6} = 2 + \frac{2}{9}$$

$$X_1 \approx 2.222222$$

برای برش دوم باید حجم تجمعی به $\frac{14}{3}$ برسد. این برش نیز در همان لایه قرار می‌گیرد و مختصات آن برابر است با:

$$X_2 = 2 + \frac{\frac{14}{3} - 1}{6} = 2 + \frac{11}{18}$$

$$X_2 \approx 2.611111$$

در نتیجه، دو برش در مختصات تقریبی $2.222222$ و $2.611111$ زده می‌شوند و هر سه قسمت حجم یکسانی شکلات خواهند داشت.

**نحوهٔ پیاده‌سازی:**

در کد دو تابع اصلی در نظر گرفتم. تابع calculateVolume ابتدا مختصات قطعات شکلات را دریافت می‌کند و مساحت مقطع هر قطعه را محاسبه می‌کند. سپس مقدار آن را به لایه‌های مربوط در آرایهٔ box اضافه کرده و در نهایت حجم کل شکلات

را برمی‌گرداند.

تابع دوم با نام findCuts با استفاده از آرایهٔ box، حجم کل شکلات و تعداد افراد، مختصات برش‌ها را محاسبه می‌کند.

در این تابع، متغیر target حجم تجمعی مورد نیاز برای هر برش را مشخص می‌کند. با استفاده از متغیرهای sum و $j$، لایهٔ مناسب پیدا شده و سپس موقعیت دقیق برش در متغیر pos محاسبه می‌شود.

در تابع main ابتدا تعداد قطعات شکلات، تعداد افراد و ابعاد کیک دریافت می‌شوند. سپس آرایهٔ box ساخته شده و دو تابع به ترتیب فراخوانی می‌شوند.

برای نگهداری حجم شکلات‌ها از نوع دادهٔ long long استفاده کردم. همچنین برای محاسبهٔ مختصات برش‌ها از نوع double استفاده شده است؛ زیرا محل برش‌ها لزوماً عدد صحیح نیست.

در نهایت، مختصات هر برش با استفاده از setprecision(12) و با دوازده رقم بعد از اعشار چاپ می‌شود تا دقت مورد نیاز مسئله رعایت شود.

**پیچیدگی زمانی و حافظه:**

برای هر قطعه شکلات، حداکثر $a$ لایه از آرایهٔ box به‌روزرسانی می‌شود. بنابراین ساخت آرایه و محاسبهٔ حجم کل شکلات‌ها در بدترین حالت $O(na)$ زمان می‌برد.

برای پیدا کردن محل برش‌ها نیز اشاره‌گر $j$ در مجموع حداکثر $a$ بار افزایش پیدا می‌کند. از طرفی، تعداد برش‌ها برابر $m-1$ است.

بنابراین پیچیدگی زمانی کل الگوریتم برابر است با:

$$\boxed{O(na+a+m)}$$

همچنین فقط یک آرایه با طول $a$ برای نگهداری حجم شکلات لایه‌ها استفاده می‌شود. در نتیجه پیچیدگی حافظهٔ اضافی برابر است با:

$$\boxed{O(a)}$$

با توجه به اینکه $a \leq 100$ است، این روش بدون نیاز به جست‌وجوی دودویی، مختصات تمام برش‌ها را با زمان و حافظهٔ مناسب محاسبه می‌کند.